

Lecture 5 — (Relational) Databases

PB138 — Basics of Web Development

The fundamentals

Concepts — Structure

Database — the actual data stored on disk: tables, rows, and the relationships between them.

DBMS (*Database Management System*) — the software that manages that data. You install, run, and connect to it. Commonly (and loosely) called "the database".

Schema — the structure of the database: which tables exist, their columns, types, and constraints. The blueprint, not the data itself.

Normalization — the process of organizing tables to reduce redundancy. More normal forms mean less duplication, sometimes at the cost of query performance.

Constraint — a rule enforced by the DBMS (`NOT NULL`, `UNIQUE`, `CHECK`, foreign keys). Lets the database guarantee data integrity so you don't have to.

Migration — a versioned script that evolves the schema over time. Think of it as a git history for your database structure.

Concepts — Operations

Transaction — a group of operations that either all succeed or all roll back together. Your guarantee of atomicity.

Index — a secondary data structure that speeds up reads at the cost of slower writes and additional storage.

View — a saved query that can be queried like a table. No extra storage — just a reusable shortcut for a complex `SELECT`.

Join — combining rows from multiple tables on a shared key. The fundamental operation for querying across related tables.

ORM — a library that maps database tables to objects in your language, abstracting away the SQL.

Primary keys

Every table needs a single column that uniquely identifies each row. Two common choices:

	UUID	serial / bigserial
Size	16 bytes	4 / 8 bytes
Sortable	No (random by default)	Yes
Guessable	No	Yes — a problem for public-facing IDs
Distributed systems	Works across instances	Collides without coordination

Recommendation: use `UUID` for web applications. Use `serial` for internal lookup tables where you control all access.

Composite primary keys are not recommended — they tend to contribute to performance issues and make constraint checks harder. Prefer a dedicated `id` column.

Other key strategies exist for specific use cases — [Snowflake IDs](#) (used by X/Twitter, Discord) encode a timestamp and machine ID into a sortable integer, making them useful for high-throughput distributed systems like messaging apps. For most web applications, UUID is sufficient.

Foreign keys and referential integrity

A foreign key references the primary key of another table. The DBMS enforces that the relationship is always valid.

```
CREATE TABLE posts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  author_id UUID NOT NULL REFERENCES users(id) ON DELETE RESTRICT  
);
```

- You cannot insert a `post` with an `author_id` that doesn't exist in `users`
- You cannot delete a `user` who has posts — the database will reject it

On delete strategies:

- `RESTRICT` — reject the delete (*safe default*)
- `CASCADE` — delete all related rows automatically (*use with care*)
- `SET NULL` — remove the reference, keep the row (*requires nullable FK*)
- Soft delete — set `deleted_at` on the record instead; related rows stay intact (*more on this shortly*)

Migrations — your schema file is not your database

Editing the schema file does not update the database. The schema describes what you *want* — a migration is the script that actually applies that change.

- Migrations are versioned SQL scripts that transition the database from one state to the next
- Each migration represents an incremental change: adding a column, renaming a table, changing a constraint
- They evolve the structure while keeping existing data intact
- Migrations must be committed to version control — the repository is the single source of truth for your database state
- Most ORMs generate migration files from your schema automatically

Keeping the schema only on a running database instance is a reliable way to lose it permanently.

Migrations — good practices

- Migration files should be timestamped and named descriptively
- Prefer additive changes — adding columns and tables is safer than dropping them
- When a column must be removed, consider a two-step approach: deprecate first, drop in a later migration
- Never edit a migration that has already been applied to a production database — create a new one instead
- When a new software version ships, migrations run against production before the new code does

Many-to-many relationships — the wrong instinct

A user can like many posts. A post can be liked by many users. First instinct: store an array of IDs in a column.

```
-- In the users table:  
liked_post_ids INTEGER[]  
  
-- In the posts table:  
liked_by_user_ids INTEGER[]
```

This breaks in multiple ways:

- No referential integrity — the database cannot verify those IDs actually exist
- Querying across them is expensive and non-standard
- Adding metadata (e.g. *when* the like happened) is impossible

Many-to-many relationships — join tables

The correct approach is a dedicated **join table** with two foreign keys.

```
CREATE TABLE user_post_likes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id),  
  post_id UUID REFERENCES posts(id),  
  created_at TIMESTAMP DEFAULT NOW(),  
  UNIQUE (user_id, post_id)  
);
```

- Referential integrity is enforced by the database
- Standard **JOIN** operations apply — efficient and portable
- The join table naturally carries additional data (timestamps, roles, status, etc.)

Most M:N relationships that feel "direct" need a join table. That table often becomes meaningful in its own right.

Transactions — what happens when only half succeeds?

Transferring money requires debiting one account and crediting another. If the credit fails after the debit succeeds, the money is gone. No error, no trace.

Transactions wrap multiple operations into a single atomic unit. Either everything commits, or everything rolls back.

```
BEGIN;  
  
UPDATE accounts SET balance = balance - 100 WHERE id = 'sender-id';  
UPDATE accounts SET balance = balance + 100 WHERE id = 'receiver-id';  
  
COMMIT;    -- both succeed: changes are persisted  
-- ROLLBACK; -- on error: neither change is applied
```

Always roll back explicitly on error. Never assume the database cleaned up after you.

Transactions — isolation levels *(bonus)*

Databases are inherently concurrent. Transactions running in parallel can interfere with each other in subtle ways. Isolation levels control the trade-off between consistency and performance.

Level	Problems
Serializable	Performance — full row locking
Repeatable reads	Serialization anomalies, phantom reads
Read committed	Non-repeatable reads, phantom reads
Read uncommitted	Dirty reads, non-repeatable reads, phantom reads

- Default to **read committed** for most operations
- Use **serializable** for anything involving money or critical shared state
- Keep transactions as short as possible — they hold locks for their entire duration

See [PostgreSQL isolation documentation](#)

Indexes — to index, or not to index

Indexes trade write performance and storage for faster reads. Like loading data into RAM — you wouldn't cache your entire dataset in memory just because it's faster to access. Storage is finite, and these days, not cheap.

Index these:

- Primary keys — done automatically by most DBMS
- Columns you frequently filter or sort by (`WHERE` , `ORDER BY`)
- Foreign key columns used in joins

Think twice before indexing:

- Columns rarely used in queries
- Tables with heavy write traffic — every `INSERT` , `UPDATE` , `DELETE` must also update the index
- Low-cardinality columns (e.g. a boolean `is_active`) — the selectivity is too low to be worth it

Rebuilding an index on a large table is expensive and impacts live performance. Profile first, index second.

Soft delete

Avoid `DELETE`. Instead, mark a record as deleted with a timestamp column.

```
-- "Deleting" a post
UPDATE posts SET deleted_at = NOW() WHERE id = '...';

-- Querying active posts
SELECT * FROM posts WHERE deleted_at IS NULL;
```

- Data stays recoverable
- Referential integrity remains intact
- Required for proper audit trails — who deleted what, and when

Soft-deleted rows still live in your indexes and count toward table size. Filter consistently — ideally at the ORM layer so no query accidentally skips it.

Pagination — returning everything is not an option

Your current API endpoint probably returns all records. That works fine with 20 rows in development. With 1 million rows in production, it will time out, exhaust memory, and bring down your API.

The naive fix: `LIMIT` and `OFFSET`.

```
-- Page 1
SELECT * FROM posts ORDER BY created_at DESC LIMIT 20 OFFSET 0;

-- Page 501
SELECT * FROM posts ORDER BY created_at DESC LIMIT 20 OFFSET 10000;
```

The problem: the database still scans and discards all preceding rows before returning your 20. Page 1 is fast. Page 500 is slow. Page 5000 is very slow — and it gets worse linearly.

Cursor-based pagination

Instead of skipping rows, use the last seen value as a filter. The database hits an index — no scanning, constant performance regardless of depth.

```
-- First page
SELECT * FROM posts ORDER BY created_at DESC LIMIT 20;

-- Next page — the cursor is the created_at of the last row from the previous page
SELECT * FROM posts
WHERE created_at < '2026-03-01T12:00:00Z' -- last seen value
ORDER BY created_at DESC LIMIT 20;
```

- Performance stays constant at any depth
- Cursor column must be **indexed** and have **high cardinality**
- No jumping to arbitrary pages — only forward/backward (*fine for feeds and infinite scroll*)

This is how X, Instagram, and GitHub's APIs handle lists and feeds.

2. Modeling

ERD — entities

An entity is a "thing" in your system worth storing data about. Each entity maps to a table, each attribute to a column.

```
entity User {  
  * id      : uuid      <<PK>>  
  ---  
  * name    : text  
  * email   : text      <<unique>>  
  * created_at : timestamp  
    deleted_at : timestamp  
}
```

- `*` — not nullable
- `---` — everything above is the primary key
- Start here before thinking about relationships — get the entities right first

ERD — data types

Pick the right type for each attribute — it affects storage, query performance, and what operations the database can do.

Type	Use for
<code>uuid</code>	Primary keys, foreign keys
<code>text</code>	Strings — no length limit needed in most cases
<code>integer</code> / <code>bigint</code>	Whole numbers
<code>numeric</code>	Exact decimals — always use for money, never <code>float</code>
<code>timestamp</code>	Date + time — <code>created_at</code> , <code>updated_at</code> , <code>deleted_at</code>
<code>boolean</code>	True/false flags

When in doubt between `text` and `varchar(n)` : prefer `text` and enforce length with a constraint if needed.

ERD — relationships

How entities connect to each other. Three kinds:

One-to-one (1:1) — one `User` has one `Profile`

Use when data is optional for most records, or large enough to keep separate.

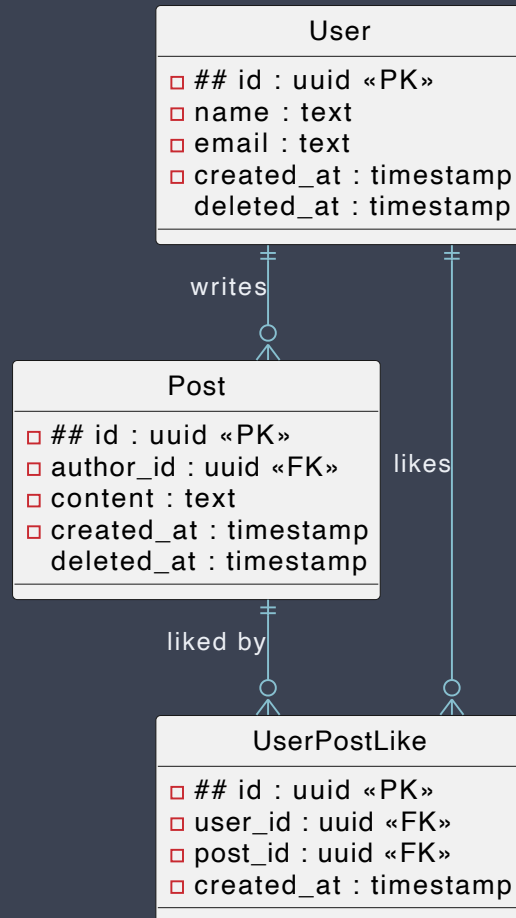
One-to-many (1:N) — one `User` has many `Post`s, a `Post` belongs to one `User`

The most common relationship. Implemented with a foreign key on the "many" side.

Many-to-many (M:N) — `User`s can like many `Post`s, `Post`s can be liked by many `User`s

Requires a join table — as we covered earlier.

ERD — putting it together



Normalization vs performance

A fully normalized schema stores each piece of data once — no redundancy, no drift. But every relationship requires a join, and joins have a cost.

Normalized — correct by default, joins on read:

```
SELECT posts.content, users.name
FROM posts JOIN users ON posts.author_id = users.id;
```

Denormalized — duplicate data to avoid the join:

```
-- author_name stored directly on the post
SELECT content, author_name FROM posts;
```

Denormalization trades write complexity for read speed. If `user.name` changes, every `post.author_name` must be updated too.

Start normalized. Denormalize only when you have a measured performance problem — not before.

3. Drizzle ORM

Raw SQL, query builders, ORMs

Three ways to talk to a database from your application code:

Approach	Pros	Cons
Raw SQL (pg)	Full control, maximum performance	No type safety, verbose, SQL injection risk
Query builder (Kysely)	Type-safe, close to SQL	No schema management, SQL knowledge still required
ORM (Drizzle, Prisma)	Schema + queries + migrations, fully type-safe	Less direct control over generated SQL

In TypeScript: Drizzle, Prisma, and TypeORM are the main options. Prisma is excellent and widely used in industry — you will likely encounter it at work. We use Drizzle because it keeps you closer to SQL, which is the more transferable skill.

Drizzle sits closer to the query builder end of this spectrum than a traditional ORM — it provides ORM-style read capabilities (relations, eager loading) but write operations remain query-builder style. This is a feature, not a limitation.

Selected technologies

PostgreSQL — the DBMS. Relational at its core, but also handles JSON, arrays, full-text search, and vectors via `pgvector`. A swiss army knife — one service covering most of what a web application needs.

`node-postgres` (`pg`) — the driver. Manages the raw TCP connection(s) between Node.js and PostgreSQL. No query building, no schema management — just sending SQL and getting results back.

Drizzle ORM — sits on top of the driver. Handles schema definition, type-safe query building, and migrations via `drizzle-kit`. Keeps you close to SQL while removing the boilerplate.

```
Your code → Drizzle → node-postgres → PostgreSQL
```

Setup

Install the required packages:

```
npm install drizzle-orm pg
npm install -D drizzle-kit @types/pg
```

Create a `.env` file with your connection string — never hardcode credentials:

```
DATABASE_URL="postgresql://user:password@localhost:5432/mydb"
```

Initialize the Drizzle client:

```
import { drizzle } from "drizzle-orm/node-postgres";
import { Pool } from "pg";

const pool = new Pool({ connectionString: process.env.DATABASE_URL });
export const db = drizzle(pool);
```

Use a `Pool`, not a single `Client` — it manages multiple connections automatically and is far better suited for a web server handling concurrent requests.

Running PostgreSQL locally

The easiest way to run PostgreSQL in development — no installation, one command:

```
# docker-compose.yml
services:
  postgres:
    image: postgres:latest
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: mydb
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

```
docker compose up -d
```

The `volumes` entry persists data between container restarts — without it, your database resets every time the container stops.

Schema definition — the pattern

Drizzle schema is TypeScript. Each table is a `pgTable` call — the ERD maps directly to code:

ERD

```
entity User {  
  * id      : uuid    <<PK>>  
  ---  
  * name    : text  
  * email   : text    <<unique>>  
  * created_at : timestamp  
    deleted_at : timestamp  
}
```

Drizzle

```
export const users = pgTable("users", {  
  // UUID generated at DB level – can also be done in app code  
  id: uuid("id").primaryKey().defaultRandom(),  
  name: text("name").notNull(),  
  email: text("email").notNull().unique(),  
  createdAt: timestamp("created_at").defaultNow().notNull(),  
  deletedAt: timestamp("deleted_at"),  
});
```

The TypeScript column names (`createdAt`) and SQL column names (`created_at`) are defined separately — use the convention that fits each side.

Schema definition — relations and constraints

Extending the schema with foreign keys and the join table:

```
export const posts = pgTable("posts", {
  id: uuid("id").primaryKey().defaultRandom(),
  authorId: uuid("author_id")
    .notNull()
    .references(() => users.id),
  content: text("content").notNull(),
  createdAt: timestamp("created_at").defaultNow().notNull(),
  deletedAt: timestamp("deleted_at"),
});

export const userPostLikes = pgTable(
  "user_post_likes",
  {
    id: uuid("id").primaryKey().defaultRandom(),
    userId: uuid("user_id")
      .notNull()
      .references(() => users.id),
    postId: uuid("post_id")
      .notNull()
      .references(() => posts.id),
    createdAt: timestamp("created_at").defaultNow().notNull(),
  },
  (t) => [unique().on(t.userId, t.postId)],
);
```

Migrations with Drizzle Kit

First, configure `drizzle-kit` — create a `drizzle.config.ts` at the project root:

```
import { defineConfig } from "drizzle-kit";

export default defineConfig({
  schema: "./src/schema.ts",
  out: "./drizzle",
  dialect: "postgresql",
  dbCredentials: { url: process.env.DATABASE_URL! },
});
```

Then generate and apply:

```
npx drizzle-kit generate    # creates SQL migration files in ./drizzle
npx drizzle-kit migrate     # applies pending migrations to the database
```

Both the TypeScript schema (`src/schema.ts`) and the generated SQL migration files (`./drizzle`) must be committed to version control. The schema is the source of truth. The migrations are the history of how you got there. Neither is optional.

Queries — reading data

```
import { eq, and, isNull, desc } from "drizzle-orm";

const userPosts = await db
  .select()
  .from(posts)
  .where(
    and(
      eq(posts.authorId, currentUser.id), // always scope to the current user
      isNull(posts.deletedAt), // exclude soft-deleted records
    ),
  )
  .orderBy(desc(posts.createdAt))
  .limit(20);
```

Drizzle generates exactly the SQL you'd write by hand:

```
SELECT * FROM posts
WHERE author_id = $1 AND deleted_at IS NULL -- $1 is parameterized, never interpreted as SQL
ORDER BY created_at DESC LIMIT 20;
```

Never query without scoping to the current user. Returning another user's private data is a security vulnerability, not just a bug.

Queries — writing data

// INSERT — returns the created record

```
const [post] = await db
  .insert(posts)
  .values({
    authorId: currentUser.id, // scope on write too — never trust client-provided user IDs
    content: "Hello world",
  })
  .returning();
```

// UPDATE — always scope to current user to prevent modifying someone else's data

```
await db
  .update(posts)
  .set({ content: "Updated content" })
  .where(and(eq(posts.id, postId), eq(posts.authorId, currentUser.id)));
```

// SOFT DELETE — an update, not a DELETE

```
await db
  .update(posts)
  .set({ deletedAt: new Date() })
  .where(and(eq(posts.id, postId), eq(posts.authorId, currentUser.id)));
```

Relations — defining them

Relations are defined separately from the schema — they are metadata for Drizzle's relational query engine:

```
export const usersRelations = relations(users, ({ many }) => ({
  posts: many(posts),
  likes: many(userPostLikes),
}));

export const postsRelations = relations(posts, ({ one, many }) => ({
  author: one(users, { fields: [posts.authorId], references: [users.id] }),
  likes: many(userPostLikes),
}));
```

`one()` = "belongs to", `many()` = "has many". The `fields / references` pair defines the join condition.

Relations are **not** part of the SQL schema — adding them generates no new migration.

Your turn: write `userPostLikesRelations` — `one()` to both `users` and `posts`.

Relations — querying with `with:` *(bonus)*

Drizzle has two query APIs. Relations unlock the **relational query API** — eager loading without writing joins manually:

You will likely see this API used in the seminars — it's worth knowing it exists.

```
// db.query — relational API (requires relations to be defined)
const userPosts = await db.query.posts.findMany({
  where: (posts, { eq, isNull, and }) =>
    and(eq(posts.authorId, currentUser.id), isNull(posts.deletedAt)),
  with: {
    author: {
      columns: { id: true, name: true }, // only select what you need
    },
    likes: true,
  },
  orderBy: (posts, { desc }) => desc(posts.createdAt),
  limit: 20,
});
```

The chainable `db.select().from()` API you've seen so far does not use `with:` — `with:` is only available via `db.query`.

Query modularity — optional filters

Real API endpoints accept optional query parameters. Instead of a separate query function for every combination, build the conditions dynamically:

```
type GetPostsParams = {
  authorId?: string; // filter by a specific author
  before?: Date; // cursor-based pagination
  limit?: number;
};

async function getPosts(currentUser: User, { authorId, before, limit = 20 }: GetPostsParams) {
  const conditions: SQL[] = [
    eq(posts.authorId, currentUser.id), // always required — access scope
    isNull(posts.deletedAt), // always required — exclude soft-deleted
  ];

  if (authorId) conditions.push(eq(posts.authorId, authorId));
  if (before) conditions.push(lt(posts.createdAt, before));

  return db.select().from(posts)
    .where(and(...conditions))
    .orderBy(desc(posts.createdAt))
    .limit(limit);
}
```

The required conditions — access scope and soft-delete filter — are always present. Optional filters are pushed in only when provided.

Think: what would happen if `conditions` ended up empty?

Transactions in Drizzle

Wrapping multiple operations in a single atomic unit:

```
await db.transaction(async (tx) => {  
  // debit  
  await tx  
    .update(accounts)  
    .set({ balance: sql`${accounts.balance} - ${amount}` })  
    .where(eq(accounts.id, senderId));  
  
  // credit  
  await tx  
    .update(accounts)  
    .set({ balance: sql`${accounts.balance} + ${amount}` })  
    .where(eq(accounts.id, receiverId));  
});
```

- `tx` is a transaction-scoped database instance — use it for **all** queries inside the callback
- If anything throws, Drizzle automatically rolls back — so your constraint checks must throw to abort (no silent failures)
- `BEGIN` and `COMMIT` are handled for you

Business logic beyond SQL

SQL constraints enforce data integrity — but not every business rule fits in a `CHECK` constraint. Some logic belongs in your application code.

Authorization — does this user own this resource?

```
const [post] = await db
  .select()
  .from(posts)
  .where(and(eq(posts.id, postId), isNull(posts.deletedAt)));

if (!post) throw new NotFoundError("Post not found");
if (post.authorId !== currentUser.id) throw new ForbiddenError("Not your post");

// safe to proceed
await db.update(posts).set({ content }).where(eq(posts.id, postId));
```

Domain invariants — rules SQL cannot express

```
// assume this runs inside a transaction
const [account] = await tx.select().from(accounts).where(eq(accounts.id, id));

if (account.balance < amount) throw new InsufficientFundsError();

// safe to proceed with transfer
```

This logic does not belong in the database layer. It belongs in the layer above it — the service layer, which we cover next.

4. Repository pattern + Service layer

Why separate layers?

Without structure, database queries end up scattered — in route handlers, in middleware, duplicated across files. Any schema change becomes a hunt.

Two layers solve this cleanly:

Layer	Responsibility
Repository	Knows how to talk to the database
Service	Knows the business rules

Route handlers call services. Services call repositories. Neither layer bleeds into the other.

Repository layer

A repository wraps all database access for one entity. No business logic here — just queries.

```
// File: src/repositories/postRepository.ts
import type { NodePgDatabase } from "drizzle-orm/node-postgres";

export function createPostRepository(db: DrizzleDb) {
  return {
    async findById(id: string) {
      const [post] = await db
        .select()
        .from(posts)
        .where(and(eq(posts.id, id), isNull(posts.deletedAt)));
      return post ?? null;
    },

    async findByAuthor(authorId: string) {
      return db
        .select()
        .from(posts)
        .where(and(eq(posts.authorId, authorId), isNull(posts.deletedAt)));
    },

    async create(data: NewPost) {
      const [post] = await db.insert(posts).values(data).returning();
      return post;
    },

    async softDelete(id: string) {
      await db
        .update(posts)
        .set({ deletedAt: new Date() })
        .where(eq(posts.id, id));
    },
  };
}
```

Service layer

The service sits above the repository. It enforces business rules before (and sometimes after) hitting the database.

```
// src/services/postService.ts

export function createPostService(
  postRepo: ReturnType<typeof createPostRepository>,
) {
  return {
    async deletePost(postId: string, currentUser: User) {
      const post = await postRepo.findById(postId);

      if (!post) throw new NotFoundError("Post not found");
      if (post.authorId !== currentUser.id)
        throw new ForbiddenError("Not your post");

      await postRepo.softDelete(postId);
    },
  };
}
```

The service doesn't know (or care) whether the repository talks to PostgreSQL, SQLite, or an in-memory map.

The abstraction leaks at transactions

The clean separation holds for single-entity operations. It gets messier when a transaction spans multiple repositories.

The pragmatic solution: **the service owns the transaction, repos accept `tx`**

```
await db.transaction(async (tx) => {  
  const accounts = createAccountRepository(tx); // pass tx, not db  
  await accounts.debit(senderId, amount);  
  await accounts.credit(receiverId, amount);  
});
```

The repository now accepts `DrizzleDb | DrizzleTransaction` — it knows about Drizzle. The abstraction leaks.

Most teams accept this tradeoff. The cleaner alternative — Unit of Work — is heavier and distinctly Java-flavoured.

Swappable by default (*bonus*)

TypeScript's structural typing means you never have to declare an interface — any object with the matching shape satisfies the type.

```
// Derive the type from the factory – stays in sync automatically
type PostRepository = ReturnType<typeof createPostRepository>;

// Real repository – used in production
const postRepo: PostRepository = createPostRepository(db);

// In-memory stub – used in tests
const postRepo: PostRepository = {
  findById: async (id) => testPosts.find((p) => p.id === id) ?? null,
  findByAuthor: async (authorId) =>
    testPosts.filter((p) => p.authorId === authorId),
  create: async (data) => {
    /* ... */
  },
  softDelete: async (id) => {
    /* ... */
  },
};

const service = createPostService(postRepo);
```

5. Seeding

Seeding — filling the database for development

A seed script populates your database with realistic data so you have something to work with locally. It is not for production — it runs once, in development.

Why bother:

- Testing with real-looking data surfaces problems that 3 hardcoded rows never will
- New team members can clone the repo and have a working local environment immediately
- Consistent data makes it easier to reproduce and debug issues

```
npx tsx src/seed.ts
```

Seed scripts are destructive by design — they usually clear existing data before inserting. Never point them at a production database.

Writing a seed script

```
// src/seed.ts
import { db } from "../db";
import { users, posts, userPostLikes } from "../schema";

async function seed() {
  // clear in reverse dependency order — children before parents
  await db.delete(userPostLikes);
  await db.delete(posts);
  await db.delete(users);

  const [alice, bob] = await db
    .insert(users)
    .values([
      { name: "Alice", email: "alice@example.com" },
      { name: "Bob", email: "bob@example.com" },
    ])
    .returning();

  const [post] = await db
    .insert(posts)
    .values([
      { authorId: alice.id, content: "Hello from Alice" },
      { authorId: bob.id, content: "Hello from Bob" },
    ])
    .returning();

  await db.insert(userPostLikes).values({ userId: bob.id, postId: post.id });
}

seed().then(() => process.exit(0));
```

For larger volumes of realistic data, reach for `faker.js` or `@snaplet/seed` — useful when you need to test pagination or performance.

6. Connecting to Express

Wiring the layers together

Everything gets composed at startup — once. Repositories get the `db` instance, services get repositories, routers get services.

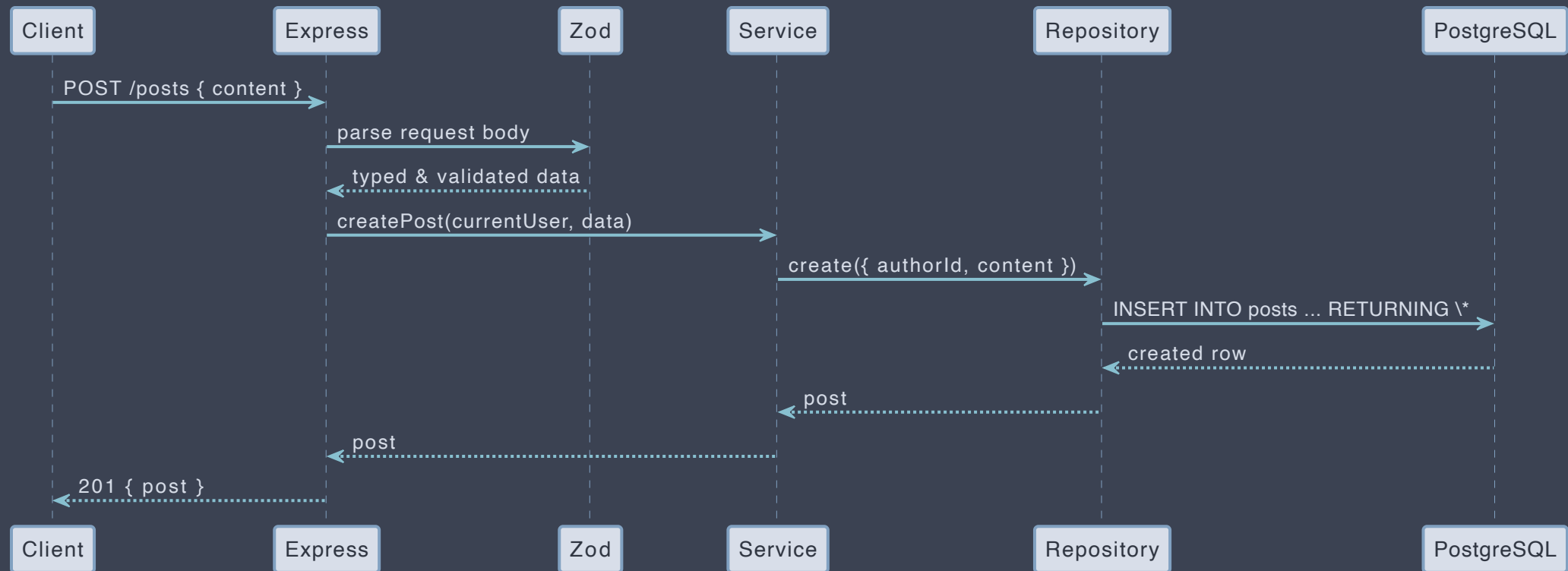
```
// src/app.ts
import { db } from "../db";
import { createPostRepository } from "../repositories/postRepository";
import { createPostService } from "../services/postService";
import { createPostRouter } from "../routes/posts";

const postRepo = createPostRepository(db);
const postService = createPostService(postRepo);

app.use("/posts", createPostRouter(postService));
```

Route handlers stay thin — no database access, no business logic. They validate input, call the service, return the result.

Full request flow



A complete route handler

```
// src/routes/posts.ts
export function createPostRouter(postService: PostService) {
  const router = Router();

  router.post("/", async (req, res) => {
    const body = createPostSchema.parse(req.body); // Zod throws on invalid input → caught by your error handler
    const post = await postService.createPost(req.user, body);
    res.status(201).json(post);
  });

  router.delete("/:id", async (req, res) => {
    await postService.deletePost(req.params.id, req.user);
    res.status(204).send();
  });

  return router;
}
```

The route handler has no knowledge of the database. It speaks HTTP — parse, delegate, respond.

Good practices

Schema design

- Use UUIDs over serial for public-facing resources
- M:N relationships always need a join table — never arrays in a column
- 1:1 is rarer than it seems — often a 1:N waiting to happen
- Start normalized — denormalize only when you have a measured problem

Queries & data access

- Always scope queries to `currentUser.id` — unscoped queries are a security bug, not just a mistake
- Soft delete over hard delete — set `deleted_at`, filter it consistently at the ORM layer
- Never return all records without pagination

Indexes

- Index primary keys (automatic), foreign keys, and columns you **regularly** filter or sort by
- Profile before adding — indexes slow **down** writes and cost storage

Everything else

- Multi-step operations belong in a transaction — constraint checks must throw to trigger rollback
- Never edit an applied migration — create a new one
- Commit both the schema file and the generated migration SQL
- Let the database do its job — constraints, filtering, ordering, and aggregation belong in SQL; reach for application code only when SQL cannot express the rule

Further reading

Drizzle ORM

- [Why Drizzle?](#)
- [Get started with Drizzle + PostgreSQL](#)

PostgreSQL

- [Indexes](#)
- [Transaction isolation](#)
- [UUID vs serial vs identity columns](#)

Pagination

- [Understanding cursor pagination and why it's so fast](#)

Design decisions

- [UUID vs serial primary keys — pganalyze](#)
- [Soft deletion probably isn't worth it — brandur.org](#) (*a counterpoint — worth reading*)
- [Snowflake IDs — Wikipedia](#)

Libraries mentioned

- [faker.js](#)
- [@snaplet/seed](#)
- [neverthrow](#)